Atelier de professionnalisation

Partie 4 : Ajout, modification, suppression des notes de frais

Application mobile de gestion des notes de frais — GSB

Introduction

Maintenant que le visiteur commercial peut s'identifier et afficher les différentes notes de frais correspondant à ses déplacements professionnels, l'application va lui permettre de les gérer.

On rappelle qu'une note de frais décrit un déplacement, saisi pour un mois précis.

Il peut y avoir plusieurs notes le même mois, pour plusieurs déplacements.

Sur chaque note, les visiteurs ont droit à deux types de frais :

- Les frais au forfait, avec un tarif fixé à l'avance (kilométrique, repas, nuit d'hôtel…) où l'on saisit des quantités.
- Les frais hors forfait, dont le coût ne peut être déterminé à l'avance (location d'une salle de conférence, prestation d'un intervenant…) où l'on saisit la date, le montant et le libellé.

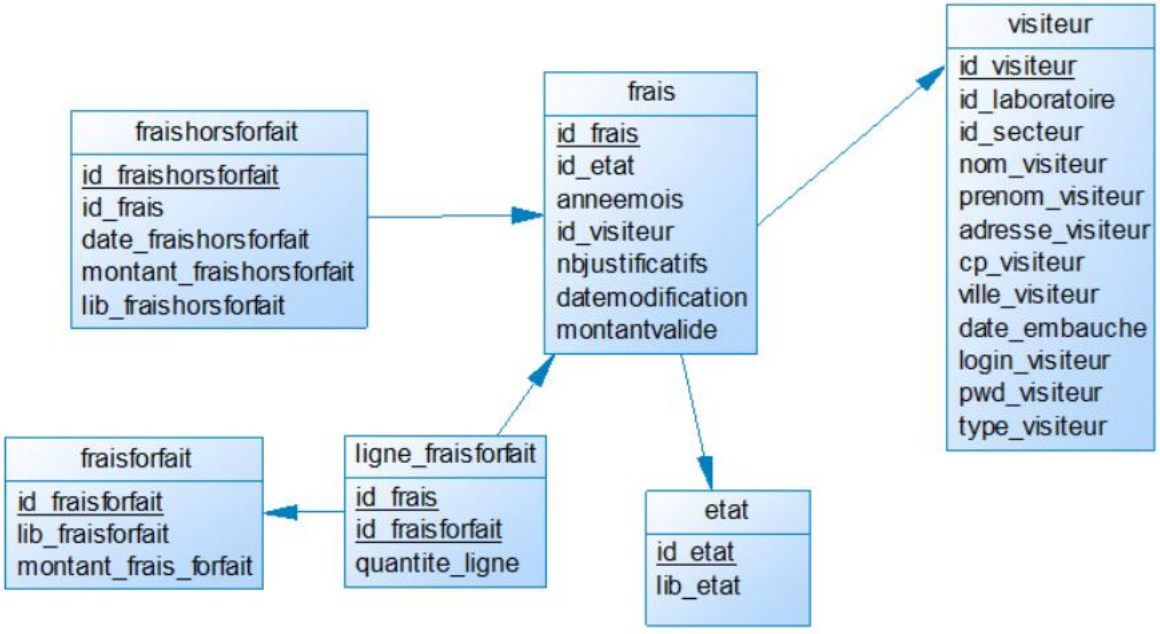
Une note de frais est donc potentiellement associée à plusieurs frais au forfait et hors forfait. Son montant est égal à la somme de tous les frais.

Lorsque le visiteur a fini de saisir une note, il doit la clôturer.

Ensuite le comptable va valider un montant à mettre en paiement en fonction des justificatifs fournis.

Puis le visiteur sera remboursé le mois suivant de tous les frais engagés.

Nous rappelons la représentation en base de données. Nous allons pour le moment nous concentrer sur la table frais, qui correspond aux notes de frais.



Nous allons compléter notre application pour en faire une application CRUD (Create-Read-Update-Delete) complète. Nous avons la fonctionnalité d'affichage (Read), il nous manque l'ajout (CREATE), la modification (UPDATE) et la suppression (DELETE).

Structure de fichiers (en gras, les fichiers à créer) :

```
config.js                <- à créer (URL de l'API)
screens/
  DashboardScreen.jsx
  FraisAddScreen.jsx     <- à créer
  FraisEditScreen.jsx    <- à créer
components/
  FraisCard.jsx
  FraisForm.jsx          <- à créer (partagé ajout/modification)
services/
  authService.js         <- à modifier (import de API_URL)
```

Méthodes et endpoints API utilisés (voir la documentation de l'API dans les ressources) :

```
GET    /api/frais/liste/{idVisiteur}
GET    /api/frais/{idFrais}
POST   /api/frais/ajout    body: { anneemois, id_visiteur, nbjustificatifs }
POST   /api/frais/modif    body: { id_frais, anneemois, id_visiteur,
nbjustificatifs, montantvalide, id_etat }
DELETE /api/frais/suppr    body: { id_frais }
```

Préparation : centraliser l'URL de l'API

Depuis la partie 3, l'adresse de l'API est écrite en dur dans les fichiers avec const API_URL=.... Les nouveaux fichiers de cette partie (FraisForm, FraisEditScreen, la suppression dans DashboardScreen) en ont aussi besoin. Plutôt que de recopier l'adresse partout, on la déplace dans un fichier de configuration, comme présenté au cours 4.

1. Créer le fichier config.js

À la racine du projet (au même niveau que App.js), créez le fichier config.js. Il exporte une seule constante : l'adresse de l'API, avec la barre oblique finale. C'est cette adresse, et uniquement celle-là, qu'il faut utiliser partout.

```
// config.js (à la racine du projet)
// export const API_URL = "http://gsbfrais.julliand.ispconfig.lmdsio.com/api/";
export const API_URL = "https://apigsbfrais.wasmer.app/api/";
```

> **⚠ Gardez la barre oblique finale**
> Dans le code cet AP, l'adresse est suivie directement du nom de la route : `${API_URL}frais/ajout`. L'URL doit donc se terminer par « / ». Sans cela, la requête part, par exemple, vers « .../apifrais/ajout » et l'API répond 404.

2. Utiliser config.js à la place de l'ancienne définition

Dans services/authService.js, supprimez la ligne qui définit API_URL et importez-la depuis config.js. Faites de même dans chaque fichier qui utilise API_URL (DashboardScreen, par exemple). Pour tous les trouver, recherchez « API_URL » dans l'ensemble du projet (Ctrl + Maj + F dans VS Code).

```
// depuis un fichier de screens/, components/ ou services/ :
import { API_URL } from "../config";

// depuis un fichier situé à la racine (App.js) :
import { API_URL } from "./config";
```

3. Vérifier que l'application fonctionne toujours

Relancez l'application, connectez-vous (avec Andre/secret) et vérifiez que la liste des frais s'affiche. Rien ne doit avoir changé : il s'agit seulement d'un déplacement de code. En cas d'erreur « Unable to resolve module », vérifiez le chemin de l'import (nombre de « ../ »).

> **Pourquoi un fichier de configuration ?**
> Quand l'adresse change (autre serveur, passage en production), il n'y a plus qu'une seule ligne à modifier, au lieu d'en chercher une dizaine dans tous les fichiers. Ce fichier est embarqué dans l'application : n'y placez jamais de mot de passe ni de clé secrète.

Ajout d'une note de frais

Étape 1 : Créer un formulaire partagé

Le composant FraisForm sera utilisé pour l'ajout ET la modification d'une note de frais.

1. Créer components/FraisForm.jsx

Ce composant contient un formulaire avec les champs anneeMois et nbJustificatifs, ainsi que l'affichage du montant saisi s'il existe (non modifiable, car calculé automatiquement) et deux boutons (pour le moment inactifs) pour ajouter des frais hors forfait et au forfait.

- Déclarez les états locaux avec useState : idFrais (null), anneeMois (''), nbJustificatifs (''), montant (''), loading (false), error ('').

```
export default function FraisForm() {
  // ToDo : déclarer les 6 états listés ci-dessus avec useState

  return (
    <View style={styles.container}>
      <Text style={styles.title}>{frais ? "Modifier la note de frais" : "Saisir
une note de frais"}</Text>

      <View>
        <Text>Année-mois :</Text>
        <TextInput
          style={styles.input}
          placeholder="Année-mois (ex: 2023-10)"
          value={anneeMois}
          onChangeText={setAnneeMois}
        />
      </View>

      <View>
        <Text>Nombre de justificatifs :</Text>
        <TextInput
          style={styles.input}
          placeholder="Nombre de justificatifs"
          keyboardType="numeric"
          value={nbJustificatifs}
          onChangeText={setNbJustificatifs}
        />
      </View>

      <View>
        <Text>Montant saisi :</Text>
        <TextInput
          editable={false}
```

```
          selectOnFocus={false}
          style={[styles.input, { backgroundColor: "#f0f0f0" }]}
          keyboardType="numeric"
          value={montant}
          onChangeText={setMontant}
        />
      </View>

      <View style={styles.smallButtonRow}>
        <Pressable
          style={[styles.smallbutton, styles.buttonDisabled]}
          //onPress={handleSubmit}
          disabled={loading}
        ><Text style={styles.buttonText}>
            Frais hors forfait
          </Text></Pressable>
        <Pressable
          style={[styles.smallbutton, styles.buttonDisabled]}
          //onPress={handleSubmit}
          disabled={loading}
        ><Text style={styles.buttonText}>
            Frais au forfait
          </Text></Pressable>

      </View>

      <Pressable
        style={[styles.button, loading && styles.buttonDisabled]}
        onPress={handleSubmit}
        disabled={loading}
      >
        <Text style={styles.buttonText}>
          {loading ? "Enregistrement..." : frais ? "Mettre à jour" : "Ajouter"}
        </Text>
      </Pressable>

      {error ? <Text style={styles.error}>{error}</Text> : null}
    </View>
  );
}
```

TextInput renvoie un String

Rappel du cours 5 : la valeur d'un TextInput est toujours une chaîne, même avec keyboardType="numeric". Il faudra convertir nbJustificatifs en nombre (parseInt) avant de l'envoyer à l'API.

2. Créer l'écran FraisAddScreen

Créez screens/FraisAddScreen.jsx, qui affiche simplement <FraisForm />. Inspirez-vous de la façon dont DashboardScreen affiche FraisCard.

Étape 2 : Gérer la soumission du formulaire

1. Implémenter handleSubmit

Complétez la fonction ci-dessous dans FraisForm.jsx. Le principe est le même qu'à la partie 3 (try/catch/finally), avec en plus la récupération du token depuis AsyncStorage.

```
const handleSubmit = async () => {
  setLoading(true);
```

```
  setError("");
  try {
    const token = await AsyncStorage.getItem("token");
    if (!token) throw new Error("Token manquant");

    const user = JSON.parse(await AsyncStorage.getItem("user"));
    const fraisData = {
      anneemois: anneeMois,
      nbjustificatifs: parseInt(nbJustificatifs, 10),
      id_visiteur: user.id_visiteur,
    };
    // ToDo : envoyer fraisData à `${API_URL}frais/ajout` avec fetch,
    // méthode POST, header Authorization Bearer token, body JSON.stringify(fraisData)

    // ToDo : naviguer vers "Dashboard" après succès
  } catch (err) {
    console.error("Erreur:", err);
    setError(err.message || "Erreur lors de l'enregistrement");
  } finally {
    // ToDo : remettre loading à false
  }
};
```

- N'oubliez pas d'importer useNavigation (pour la redirection), AsyncStorage et API_URL (depuis config.js) en haut du fichier.

2. Ajouter une vérification de response.ok

Sans cette vérification, fetch ne lève pas d'erreur sur une réponse 400 (requête incorrecte, données invalides ou mal formatées) ou 401 (accès non autorisé, token expiré), et le formulaire passe au Dashboard comme si l'ajout avait réussi. En effet, fetch ne lève une erreur que si la requête n'arrive pas au serveur (pas de réseau, adresse injoignable).
Compléter le code suivant et l'intégrer au bon endroit pour qu'en cas d'échec de la requête d'ajout à l'API, une erreur soit levée (avec throw) en reprenant le message error renvoyé par l'API, ou à défaut le code http :

1. `const data = await response.json().catch(() => ({}));`
   - response.json() lit le corps de la réponse et le transforme en objet JavaScript. On espère que le serveur y a mis un message, par exemple { "error": "Mois déjà saisi" }.
   - .catch(() => ({})) sert de sécurité : en cas d'erreur, le serveur renvoie parfois une page HTML ou rien du tout, et .json() planterait. Dans ce cas, on récupère un objet vide {} à la place.
   - Les parenthèses dans ({}) sont nécessaires. Sans elles, JavaScript lirait {} comme un bloc de code vide et non comme un objet.
2. `throw new Error(data.error || `Erreur HTTP ${response.status}`);`
   - throw interrompt le try et fait sauter directement au catch (err). La navigation vers le Dashboard n'est donc jamais exécutée.

```
// response.ok vaut true si le code http est entre 200 et 299 (succès)
if (.....) { // ToDo : si response.ok n'est pas true
1  const data = await response.json().catch(() => ({}));

2  throw new Error(data.error || `Erreur HTTP ${response.status}`);
}
```

Dans le catch, setError(err.message) enregistre ce message dans l'état. On affiche alors le message d'erreur juste au dessus du bouton Ajouter :

```
{error ? <Text style={{ color: "red" }}>{error}</Text> : null}
```

Ensuite, le finally remet loading à false.

Étape 3 : Configurer la navigation et tester

1. Ajouter l'écran à la stack

Ajoutez un Stack.Screen name="FraisAdd" component={FraisAddScreen} dans votre Stack.Navigator dans App.js, qui ne doit être présent que si l'utilisateur est connecté.

Attention, vous ne pouvez pas mettre deux composants <Stack.Screen> à la suite dans les englober dans un wrapper, cela produirait une erreur de syntaxe car le test conditionnel ne doit renvoyer qu'une valeur. Il faut donc suivre la syntaxe suivante :

```
<>
      <Stack.Screen … />
      <Stack.Screen … />
</>
```

2. Ajouter le bouton dans la Navbar

Ajoutez un bouton « Proposer un frais » dans la Navbar qui navigue vers FraisAdd (uniquement si l'utilisateur est connecté). Comme ci-dessus, vous devez englober les deux Pressable dans un wrapper.

3. Tester la nouvelle fonctionnalité

Testez : vérifiez que le frais est bien ajouté dans la base et que la navigation vers Dashboard fonctionne après l'ajout.

4. Régler le « bug »

La liste ne se met pas à jour ? Après un ajout (et de même après une modification), navigate("Dashboard") revient sur l'écran Dashboard qui était resté monté dans la pile : son useEffect ne se relance pas, la liste reste ancienne.

Rappel du cours 3 : useFocusEffect s'exécute à chaque retour sur l'écran.

Remplacez le useEffect de DashboardScreen par le code ci-dessous avec useFocusEffect : le contenu de l'ancien useEffect se place dans useCallback, avec les mêmes dépendances.

```
import { useCallback } from "react";
import { useFocusEffect } from "@react-navigation/native";

useFocusEffect(
  useCallback(() => {
    // ToDo : déplacer ici la fonction fetchFrais et son appel
  }, [user, token])
);
```

Étape 4 : Mettre en forme

Créer un fichier fraisFormStyles.js pour styler les champs et le bouton (voir les styles déjà écrits pour LoginScreen dans l'AP1, la logique est identique).

Modification d’une note de frais

Nous souhaitons maintenant permettre à l’utilisateur de modifier les informations d’une note de frais, en cliquant dessus directement dans la liste affichée dans le Dashboard.

Étape 1 : Ajouter un bouton de modification

1. Ajouter le bouton dans FraisCard

Nous allons ajouter le bouton Modifier directement dans la carte, sous forme de Pressable.

L'idée est de séparer le composant qui affiche de celui qui décide. FraisCard affiche et les comportements de boutons seront implémentés dans Dashboard : handleEdit, puis handleDelete dans la suite. FraisCard ne sait pas ce qui se passe quand on appuie sur Modifier. Elle signale seulement « on a appuyé sur Modifier pour le frais 42 », et c'est le Dashboard qui choisit quoi faire.

```
// ToDo : importer Pressable

// La carte ne fait qu'afficher : les actions sont fournies par le parent Dashboard
function FraisCard({ frais, onEdit }) {
  const navigation = useNavigation();
  return (
    <View style={styles.card}>
      {/* ... affichage existant ... */}
      <Pressable
        // ToDo : onPress doit appeler onEdit avec l’argument frais.id_frais
        style={styles.editButton} // ToDo : implémenter le style editButton
      >
        <Text>Modifier</Text>
      </Pressable>
    </View>
  );
}
```

2. Implémenter le comportement du clic sur le bouton Modifier

Le clic sur le bouton Modifier doit rediriger vers l’écran FraisEdit, que nous ajouterons à la Stack dans l’étape suivante.

navigate(nom, params) accepte deux arguments : le nom de l’écran de destination, identique au name déclaré dans Stack.Screen, puis, de façon facultative, un objet JavaScript contenant les données à transmettre à cet écran.

Sur le web, on placerait l’identifiant dans l’URL (par exemple /frais/12/edit). Il n’y a pas d’URL sur mobile : les données voyagent dans les paramètres. L’écran d’arrivée les récupère avec useRoute(), qui renvoie l’objet route : ses paramètres sont dans route.params. Voici un exemple de navigation vers une page de profil depuis un Dashboard :

```
// écran de départ
navigation.navigate("Profil", { userId: 12, from: "Dashboard" });

// écran d’arrivée (Profil)
const route = useRoute();
const { userId, from } = route.params;
```

Que faut-il transmettre ? React Navigation recommande des paramètres minimaux : des identifiants, pas des objets de données complets. C’est l’écran d’arrivée qui va chercher les données dont il a besoin.

Ici, il suffit de passer l'id de la note de frais à modifier, pas le frais complet. Ajouter le code suivant dans DashboardScreen :

```
// Ouverture du formulaire de modification
function handleEdit(id){
   navigation.navigate("FraisEdit", { id });
};
```

Puis, dans le return de DashboardScreen, passer la fonction handleEdit à l'enfant FraisCard, qui la reçoit dans sa prop onEdit :

```
<Flatlist
   ...
   renderItem={({ item }) => <FraisCard frais={item} onEdit={handleEdit} />}
   ...
/>
```

Étape 2 : Configurer la navigation

Ajoutez un Stack.Screen name="FraisEdit" component={FraisEditScreen} à votre Stack navigator dans App.js.

Étape 3 : Implémenter FraisEditScreen

FraisEditScreen est un composant « intermédiaire » : il récupère les données du frais à modifier et les transmet à FraisForm. Relation parent (FraisEditScreen, gère la récupération API) / enfant (FraisForm, gère l'affichage et la soumission).

```
import { useState, useEffect } from "react";
import { useRoute } from "@react-navigation/native";
import AsyncStorage from "@react-native-async-storage/async-storage";
import { API_URL } from "../config";
import FraisForm from "../components/FraisForm";

export default function FraisEditScreen() {
  const route = useRoute();
  const { id } = route.params; // récupération de l'id passé en paramètre de navigate()
  const [frais, setFrais] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchFrais = async () => {
      try {
        const token = await AsyncStorage.getItem("token");
        // ToDo : appeler `${API_URL}frais/${id}` avec fetch et le header Authorization
        // ToDo : comme dans FraisForm, lever une erreur si la réponse n'est pas ok
        // ToDo : mettre à jour l'état frais avec la réponse
      } catch (error) {
        console.error("Erreur:", error);
      } finally {
        setLoading(false);
      }
    };
    fetchFrais();
  }, [id]);

  if (loading) return <ActivityIndicator size="large" />;
  if (!frais) return <Text>Frais non trouvé</Text>;

  return <FraisForm frais={frais} />;
}
```

Étape 4 : Adapter FraisForm pour la modification

1. Recevoir et utiliser la prop frais

FraisForm doit maintenant accepter une prop frais, modifiez la définition du composant pour ajouter cette prop :

```
export default function FraisForm({frais=null}) {
    ...
}
```

Si frais est fourni, pré-remplissez le formulaire avec un useEffect déclenché à chaque changement de la prop.

```
useEffect(() => {
  if (frais) {
    setIdFrais(frais.id_frais);
    setMontant(frais.montantvalide || "");
    // ToDo : compléter en pré-remplissant aussi anneeMois et nbJustificatifs
    // Indice : la valeur d'un TextInput est une chaîne. L'API renvoie
    // nbjustificatifs sous forme de nombre : convertissez-le avec String(...).
  }
}, [frais]);
```

2. Distinguer ajout et modification dans handleSubmit

Nous allons utiliser la même fonction handleSubmit pour l'ajout et la modification d'une note de frais. Mais le corps de la requête diffère selon le cas : pour l'ajout on a (id_visiteur, anneemois, nbjustificatifs) alors que pour la modification on a (id_frais, anneemois, nbjustificatifs, montantvalide).

Complétez le bloc if/else, en vous appuyant sur le tableau des endpoints en début de document ou la documentation de l'API GSB.

```
// …code de handleSubmit jusqu'à la déclaration de fraisData
const fraisData = {
  anneemois: anneeMois,
  id_visiteur: user.id_visiteur,
  nbjustificatifs: parseInt(nbJustificatifs, 10),
};

let url, body;
if (frais) {
  // Modification (UPDATE)
  fraisData.id_frais = idFrais;
  fraisData.montantvalide = parseFloat(montant);
  fraisData.id_etat = … ; // ToDo : on conserve l'état du frais enregistré
  // ToDo : url = ... (endpoint de modification, voir tableau du début)
} else {
  // Ajout (CREATE)
  // ToDo : url = ... (endpoint d'ajout)
}

const response = await fetch(url, {
  method: "POST",
  headers: { "Content-Type": "application/json", Authorization: `Bearer ${token}`
},
  body: JSON.stringify(fraisData),
});
```

3. **Adapter les textes affichés**

   Remplacer le code du titre et du texte du bouton par les codes ci-dessous pour qu'ils reflètent le mode ajout ou modification.

```
<Text style={styles.formTitle}>{frais ? "Modifier la note de frais" : "Saisir une
note de frais"}</Text>

<Pressable
  style={[styles.button, loading && styles.buttonDisabled]}
  onPress={handleSubmit}
  disabled={loading}
>
  <Text>
    {loading ? "Enregistrement..." : /* ToDo : "Mettre à jour" ou "Ajouter" selon
le cas */ ""}
  </Text>
</Pressable>
```

4. **Tester**

   Vérifiez que les données sont bien envoyées à l'API et modifiées dans la base, et que la redirection fonctionne après modification.

Suppression d'une note de frais

Étape 1 : Ajouter un bouton de suppression

Comme pour la modification, ajoutez un bouton Supprimer dans FraisCard, qui appellera onDelete qui déclenchera l'exécution de handleDelete défini dans DashboardScreen.

Étape 2 : Gérer la confirmation et la suppression

> **⚠ Pas de window.confirm en React Native**
> Sur le web, window.confirm() ouvre une pop-up bloquante avec OK/Annuler. Ça n'existe pas sur mobile : on utilise Alert.alert(titre, message, boutons), où boutons est un tableau d'objets { text, onPress, style }.

Il faut implémenter handleDelete dans DashboardScreen (qui gère fraisList) et la transmettre à FraisCard en prop.

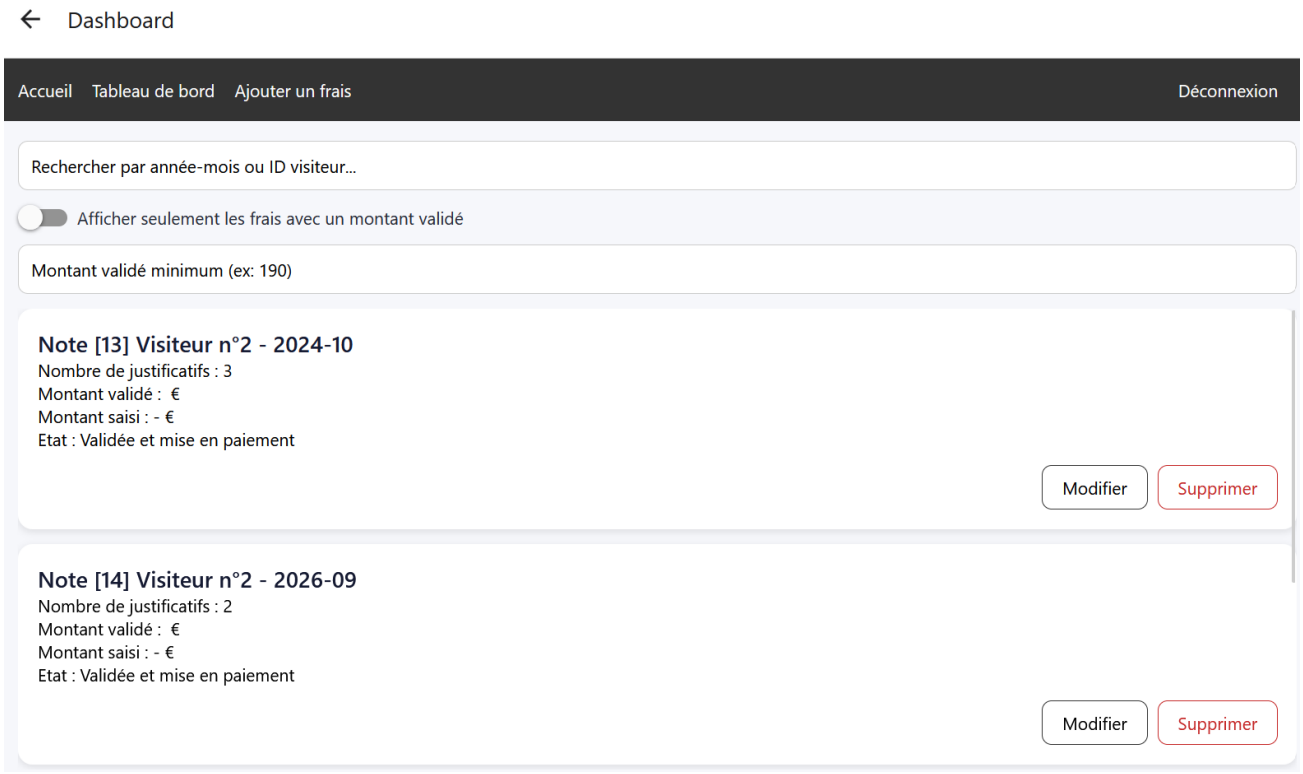
```
const handleDelete = (id) => {
  Alert.alert(
    "Confirmation",
    "Êtes-vous sûr de vouloir supprimer cette note de frais ?",
    [
      { text: "Annuler", style: "cancel" },
      {
        text: "Supprimer",
        style: "destructive",
        onPress: async () => {
          try {
            const token = await AsyncStorage.getItem("token");
            // ToDo : appeler `${API_URL}frais/suppr` avec fetch, méthode DELETE,
            // body JSON.stringify({ id_frais: id }), header Authorization

            // ToDo : mettre à jour fraisList en filtrant la note de frais
supprimée
          } catch (error) {
            console.error("Erreur lors de la suppression:", error);
```

```
            }
          },
        },
      ]
    );
  };
```

- Testez la suppression : vérifiez qu'Annuler ne fait rien, et que Supprimer retire bien le frais de la liste affichée ET de la base de données.

Exemple d'affichage du tableau de bord après avoir ajouté deux nouveaux frais à David (utilisateur Andre/secret) :



Conclusion

Vous avez maintenant une application CRUD complète sur mobile, permettant de gérer des notes de frais.

Ressources

↗ Alert — API complète

↗ AsyncStorage — documentation

↗ React Navigation — Passer des paramètres à un écran

↗ React Navigation — Group (Stack.Group, navigationKey)

↗ React Navigation — Flux d'authentification

↗ OWASP — Cross-Site Request Forgery (CSRF)